개발자·관리자 가이드

적심·적과 농작업 로봇 관제 시각화 시뮬레이터

목차

1. 본 가이드의 목적과 대상 독자

1.1 범위

2. 아키텍처 요약

2.1 모듈 계층

2.2 데이터 흐름

3. 빌드 환경

3.1 필수 도구

3.2 권장 도구

3.3 의존성 설치

4. 빌드·배포 명령

4.1 개발 모드

4.2 프로덕션 빌드

4.3 빌드 결과 패키징

4.4 정적 호스팅 배포

4.5 타입체크 단독

5. 시나리오 추가 절차

5.1 시나리오 정의 추가

5.2 타입 확인

5.3 Picker 노출 확인

5.4 검증

6. 디텍터 교체 절차

6.1 Detector interface 확인

6.2 신규 디텍터 구현

6.3 등록

6.4 검증

7. 카메라 스트림 프로토콜 (개발자 수준)

7.1 WS 엔드포인트

7.2 프레임 포맷

7.3 Producer 직접 구현 예

7.4 Consumer 직접 구현 예

7.5 Relay 동작

8. Web Worker 디텍션 파이프라인 디버그

8.1 Worker 위치

8.2 Worker 로그 활성
8.3 Back-pressure 동작
8.4 Worker terminate
8.5 ONNX 모델 로드 확인

9. 로깅 시스템 사용법

9.1 logger 가져오기
9.2 namespace 12종
9.3 사용 패턴
9.4 활성화
9.5 DevTools helper

10. 디버그 옵션

10.1 추가 globals

11. 좌표계 규약 (요약)

11.1 4개 좌표계
11.2 실수하기 쉬운 트랩
11.3 좌표 변환 utility

12. Mesh Anchor 규약 (요약)

12.1 anchor helper
12.2 새 mesh type 추가 시

13. 테스트 운영

13.1 두 종류의 테스트
13.2 전체 테스트 실행
13.3 특정 probe만 실행
13.4 Probe 추가 시
13.5 로그 enforcement

14. 모델 업데이트 절차

14.1 ONNX 모델 교체
14.2 메타데이터 갱신
14.3 디텍터 코드 동기화
14.4 HSV 파라미터 갱신
14.5 검증

15. 트러블슈팅 (개발자 관점)

15.1 npm run build 실패
15.2 dev server가 port 8090 점유 충돌
15.3 ONNX 모델 로드 실패

15.4 WS 연결 실패

15.5 디텍션이 0건만 나옴

16. 코드 컨벤션 / 협업 흐름

16.1 코드 컨벤션

16.2 Pull Request 흐름

16.3 핵심 SSOT 문서

1. 본 가이드의 목적과 대상 독자

본 가이드는 운영자 관점의 매뉴얼(`07_User_Manual.pdf`)과 달리, **본 시뮬레이터의 빌드·배포·확장·유지보수를 담당하는 개발자 또는 관리자**를 대상으로 한다.

1.1 범위

- 빌드 환경 구성 및 명령
- 시나리오 추가 절차
- 디텍터 교체 절차
- 카메라 스트림 프로토콜 (개발자 수준)
- Web Worker 디텍션 파이프라인 디버그
- 로깅 시스템 사용법
- 디버그 옵션
- 좌표계 및 Mesh Anchor 규약
- 테스트 운영
- 모델 업데이트 절차
- 트러블슈팅
- 코드 컨벤션 및 협업 흐름

2. 아키텍처 요약

상세는 `06_Architecture_Spec.pdf` 참조. 본 가이드는 운영·유지보수 관점에서 핵심만 요약한다.

2.1 모듈 계층

1. **UI Layer** — React + Zustand 상태관리
2. **Phenotyping Domain** — `src/phenotyping/` 트리 (domain / streaming / ui)
3. **Scene Layer** — Babylon.js 기반
4. **Plant Engine** — `packages/tomato-engine` (engine-pure)
5. **Tooling** — Vite plugin (`tooling/vite/plugins/camStream.ts`)
6. **Test Infrastructure** — `tests/architecture/`

2.2 데이터 흐름

```
Scenario JSON → SceneInfrastructure → Babylon Scene → Gimbal Camera RTT
 → WS Producer → WS Relay → 2 Consumers (PiP + Detection Worker)
 → Detection Worker → ONNX/HSV → Main Thread (raycast) → twinStore
 → SVG Overlay (GimbalDetectionOverlay)
```

3. 빌드 환경

3.1 필수 도구

도구	버전
Node.js	20 LTS 이상
npm	10 이상
Git	2.40 이상
운영체제	Windows 11 / macOS 14 / Ubuntu 22.04

3.2 권장 도구

도구	용도
VSCode	코드 편집 + TS 지원
Chrome DevTools	디버깅
Playwright	테스트 실행

3.3 의존성 설치

```
git clone <repo-url> MonitoringSim
cd MonitoringSim
npm install
```

`package-lock.json` 이 함께 제공되므로 동일 버전의 의존성이 설치된다.

4. 빌드·배포 명령

4.1 개발 모드

```
npm run dev
```

Vite dev server가 `http://localhost:8090`에서 시작한다. HMR 활성.

WS relay plugin (`tooling/vite/plugins/camStream.ts`)도 같은 포트에 mount된다.

4.2 프로덕션 빌드

```
npm run build
```

내부적으로 다음을 수행한다.

```
tsc --noEmit # 타입 체크 (실패 시 빌드 중단)
vite build # 정적 자산 생성 → dist/
```

`dist/` 폴더가 결과물이다.

4.3 빌드 결과 패키징

```
cd dist
zip -r ../monitoring_sim_build.zip .
```

압축 해제 후 정적 호스팅이 가능하다.

4.4 정적 호스팅 배포

방법	명령
`serve`	`npx serve dist`
Python	`cd dist && python3 -m http.server 8090`
Nginx	`dist/` 를 document root로 설정
VSCode Live Server	`dist/index.html` 을 열고 우클릭 → "Open with Live Server"

4.5 타입체크 단독

```
npm run tsc:engine
npm run tsc:geometry
npx tsc --noEmit
```

각각 packages/tomato-engine, packages/tomato-geometry, 전체 프로젝트 타입체크.

5. 시나리오 추가 절차

새 시나리오를 추가하려면 다음 단계를 따른다.

5.1 시나리오 정의 추가

`src/scenarios/loader.ts`에 새 시나리오 ID와 ScenarioSpec을 등록한다.

```
const SCENARIOS: Record<string, ScenarioSpec> = {
 'phenotyping-D110-survey': { ... },
 // 신규
 'phenotyping-D85-survey': {
 id: 'phenotyping-D85-survey',
 domain: 'phenotyping',
 crop: { day: 85, seed: 20260520, cultivar: 'tomimaru' },
 world: { bedLayout: '2-4-4', activeBedIds: [4,5,6,7,8,9] },
 robot: { profile: 'phenotyping', traverseEnabled: true, gimbalView: true },
 task: { rule: 'left-bed-on-forward,right-bed-on-return', speedMps: 0.3 },
 env: { lightingPreset: 'noon-bright', manualHour: 12 },
 },
};
```

5.2 타입 확인

```
npx tsc --noEmit
```

`ScenarioSpec` 구조와 맞는지 확인.

5.3 Picker 노출 확인

시나리오는 `Picker` UI에 자동으로 노출된다 (`SCENARIOS` 객체에 추가되면 즉시 표시). 별도 등록 절차 없음.

5.4 검증

다음 URL로 직접 진입해서 동작 확인.

```
http://localhost:8090/?mode=workbench&scenario=phenotyping-D85-survey
```

6. 디텍터 교체 절차

본 시뮬레이터는 plug-and-play 방식의 디텍터 인터페이스를 제공한다. 새 디텍터를 추가하려면 다음 단계를 따른다.

6.1 Detector interface 확인

`src/phenotyping/domain/detectors/types.ts` :

```
export interface Detector {
 id: DetectorId;
 label: string;
 source: DetectorSource;
 detect(panorama: ImageData, ctx: DetectorContext): Promise<FruitDetection[]>;
 init?(onProgress?: (loaded: number, total: number) => void): Promise<void>;
 dispose?(): void;
}
```

6.2 신규 디텍터 구현

`src/phenotyping/domain/detectors/myDetector.ts` :

```
import type { Detector, DetectorContext, FruitDetection } from './types';

export const myDetector: Detector = {
 id: 'my-v1',
 label: 'My Custom Detector',
 source: 'custom',
 async init(onProgress) {
 // 모델 로드 등
 },
 async detect(panorama, ctx) {
 // 핵심 detection logic
 return [/* ... */];
 },
 dispose() {
 // cleanup
 },
};
```

6.3 등록

`src/phenotyping/domain/detectors/index.ts` 의 `ALL_DETECTORS` 배열에 추가.

```
import { myDetector } from './myDetector';

export const ALL_DETECTORS: readonly Detector[] = [
 hsvDetector,
 onnxYoloDetector,
 groundTruthDetector,
 myDetector, // 신규
];
```

`DetectorId` 타입 union에 새 ID를 추가.

6.4 검증

브라우저에서 디텍터 셀렉터로 새 디텍터 선택 후 서베이 실행.

7. 카메라 스트림 프로토콜 (개발자 수준)

7.1 WS 엔드포인트

```
ws://<host>/robot/<streamId>?role=producer
ws://<host>/robot/<streamId>?role=consumer
```

- 기본 streamId: `phenotyping/gimbal`
- producer 1 ↔ consumer N
- 인증 없음 (localhost 전용)

7.2 프레임 포맷

매 프레임마다 두 메시지 쌍:

1. Text 메시지 — JSON FramePoseHeader
2. Binary 메시지 — JPEG bytes

`src/phenotyping/streaming/streamProtocol.ts` 참조.

7.3 Producer 직접 구현 예

```
const ws = new WebSocket('ws://localhost:8090/robot/phenotyping/gimbal?role=producer');
ws.onopen = () => {
 const header = { ts, railX, panRad, /* ... */ };
 ws.send(JSON.stringify(header));
 ws.send(jpegArrayBuffer);
};
```

7.4 Consumer 직접 구현 예

```
const ws = new WebSocket('ws://localhost:8090/robot/phenotyping/gimbal?role=consumer');
let pendingHeader = null;
ws.onmessage = (ev) => {
 if (typeof ev.data === 'string') {
 pendingHeader = JSON.parse(ev.data);
 } else {
 const blob = new Blob([ev.data], { type: 'image/jpeg' });
 // ... 사용
 }
};
```

7.5 Relay 동작

`tooling/vite/plugins/camStream.ts`:

- Vite HTTP server의 `upgrade` 이벤트에 핸들러 mount
- `/robot/<streamId>` path만 인식
- producer 메시지를 모든 active consumer에 fanout
- producer disconnect 시 모든 consumer에 `{__event: 'producer-detached'}` 알림

8. Web Worker 디텍션 파이프라인 디버그

8.1 Worker 위치

```
src/phenotyping/streaming/detectionWorker.ts — Worker
src/phenotyping/streaming/detectionWorkerClient.ts — Main thread
```

8.2 Worker 로그 활성

브라우저 DevTools Console에서:

```
__farmsim.debug.enable('scenarios')
location.reload()
```

또는 URL에 `?debug=scenarios` 추가. Worker 로그는 `[scenarios]` prefix로 표시.

8.3 Back-pressure 동작

이전 추론이 진행 중이면 새 프레임은 드롭된다. 드롭 카운트는 `runPerFrameDetect dropped N frames` 로그에 표시.

8.4 Worker terminate

서베이 중지 또는 RESET 시 worker가 `worker.terminate()` 로 종료된다. 새 서베이 시작 시 새 worker가 spawn된다.

8.5 ONNX 모델 로드 확인

Worker 첫 init 시 `detectorLoadProgress` (twinStore.phenotypingSurvey) 값을 0-1 사이로 업데이트한다. TaskPanel UI에서도 progress 표시.

9. 로깅 시스템 사용법

9.1 logger 가져오기

```
import { createLogger } from './utils/logger';
const log = createLogger('engine');
```

9.2 namespace 12종

`engine` · `scene` · `quality` · `progressive` · `skinplant` · `overlay` · `growth` · `leaf` · `plant` · `scenarios` · `workbench` · `ui`

9.3 사용 패턴

```
log.debug('detail trace'); // opt-in only
log.info('first frame rendered'); // default silent (warn 이상에서만)
log.warn('init failed', err); // 항상 출력
log.error('fatal', err); // 항상 출력
```

9.4 활성화

URL `?debug=engine,scene` (CSV) 또는 `?debug=*` (모든 namespace).

9.5 DevTools helper

```
__farmsim.debug.enable('scenarios')
__farmsim.debug.enableAll()
__farmsim.debug.disable()
__farmsim.debug.silence('growth')
__farmsim.debug.current()
```

10. 디버그 옵션

URL 파라미터로 디버그 모드를 활성화.

파라미터	의미
`?debug=stream`	스트리밍 producer/consumer 로그
`?debug=capture`	`window.__lastCaptureFrame` 에 마지막 캡처 노출
`?debug=isolatedLeaf`	잎 단일 디버그 모드
`?debug=scenarios`	시나리오 디버그
`?debug=*`	모든 namespace
`?quality=1` - `10`	렌더 품질 강제
`?seed=N`	시드 오버라이드

10.1 추가 globals

DevTools Console에서 접근 가능:

변수	의미
`__twinStore`	전역 store
`__farmsim.scene`	Babylon scene 객체
`__farmsim.engine`	Babylon engine
`__farmsim.debug`	로깅 helper
`__lastCaptureFrame`	가장 최근 RTT 캡처 (`?debug=capture` 활성 시)

11. 좌표계 규약 (요약)

상세는 `06_Architecture_Spec.pdf` 의 "좌표계 규약" 섹션 또는 `docs/architecture/COORDINATE_SYSTEMS.md` 참조.

11.1 4개 좌표계

좌표계	suffix 예시
world	`worldPos` , `worldX` , `worldY`
plant-local (lushGroup 자식)	`plantPos` , `plantLocalX`
mesh-local (vertex 안)	`meshPos` , `meshLocalX`
graph (= plant-local 별칭)	`graphPos` (SkeletonNode/Edge)

11.2 실수하기 쉬운 트랩

- `mesh.position` 은 **parent-local** (plant-local), `mesh.absolutePosition` 이 world.
- `mesh.position` 또는 `mesh.rotation` 설정 직후 `getWorldMatrix()` = identity. 반드시 `computeWorldMatrix(true)` 호출.
- `boundingBox.minimum` 은 가상 corner — 실제 vertex 아님. 의미점은 `getVerticesData('position')` 직접 query.
- 회전 origin = mesh-local `(0, 0, 0)` — vertex shift 의무.

11.3 좌표 변환 utility

`src/plant/coordinates/transforms.ts` :

- `worldToPlantLocal`
- `plantLocalToWorld`
- `meshLocalToWorld`
- `meshLocalToPlantLocal`

inline `Vector3.TransformCoordinates(...)` 직접 호출 금지.

12. Mesh Anchor 규약 (요약)

상세는 `06_Architecture_Spec.pdf` 또는 `docs/architecture/MESH_ANCHORS.md`.

12.1 anchor helper

`src/plant/anchors/leafAnchor.ts`:

- `normalizeLeafMeshVertices`
- `makeLeafAnchor`
- `assertLeafAnchorInvariant`

12.2 새 mesh type 추가 시

1. `docs/architecture/MESH_ANCHORS.md`에 contract 추가
2. `src/plant/anchors/`에 helper 추가
3. invariant test 추가

13. 테스트 운영

13.1 두 종류의 테스트

종류	위치	책임
Invariant	`tests/architecture/*.spec.ts` (probe 아닌 것)	영구 계약, CI 차단
Probe	`tests/architecture/_probe-*.spec.ts`	진단 도구, 일회용 가능

13.2 전체 테스트 실행

```
npx playwright test
```

13.3 특정 probe만 실행

```
npx playwright test tests/architecture/_probe-stream-pipeline.spec.ts
```

13.4 Probe 추가 시

1. `tests/architecture/_probe-<topic>.spec.ts` 작성
2. 조사 종료 시 `tests/architecture/PROBES.md` 정책에 따라 keep/archive 결정
3. archive 대상은 `tests/architecture/_archive/` 로 이동

13.5 로그 enforcement

`tests/architecture/logger-enforcement.spec.ts` 가 production source에서 직접 `console.*` 호출 0건을 검증. CI 통과 조건.

14. 모델 업데이트 절차

14.1 ONNX 모델 교체

```
# 1. 새 모델 파일을 public/models/yolov8n.onnx로 교체
cp <new_model>.onnx public/models/yolov8n.onnx

# 2. SHA-256 계산
shasum -a 256 public/models/yolov8n.onnx
# 결과를 03_Detection_Models 패키지의 yolov8n.onnx.sha256 갱신
```

14.2 메타데이터 갱신

`04_Detection_Models.zip` 내부의 `model_metadata.json`:

- `sha256` 필드 갱신
- `file_size_bytes` 갱신
- 입출력 텐서 사양 변경 시 해당 필드 갱신
- 클래스 매핑 변경 시 `class_mapping.accepted_classes` 갱신

14.3 디텍터 코드 동기화

`src/phenotyping/domain/detectors/onnxYoloDetector.ts`:

- `MODEL_URL` 경로 확인
- `INPUT_SIZE`, `CONF_THRESHOLD`, `IOU_THRESHOLD`, `FRUIT_CLASS_IDS` 새 모델에 맞게 조정

14.4 HSV 파라미터 갱신

`src/phenotyping/domain/detectors/hsvDetector.ts`의 `BANDS`, `MIN_BLOB_AREA`, `ASPECT_MIN`, `ASPECT_MAX` 등을 새 데이터에 맞게 재튜닝하고, `hsv_parameters.json` 갱신.

14.5 검증

```
npx playwright test tests/architecture/_probe-detect-flow.spec.ts
```

15. 트러블슈팅 (개발자 관점)

15.1 `npm run build` 실패

- `npx tsc --noEmit` 단독 실행해서 타입 오류 확인
- `dist/` 폴더가 잠겨 있는지 확인 (다른 프로세스가 열고 있을 수 있음)

15.2 dev server가 port 8090 점유 충돌

- 다른 프로세스가 점유 중 — `lsof -i :8090` 로 확인
- 또는 vite config의 port를 변경 (`vite.config.ts`)

15.3 ONNX 모델 로드 실패

- `public/models/yolov8n.onnx` 파일 존재 확인
- SHA-256 일치 확인
- 브라우저 Network 탭에서 모델 GET 응답 확인

15.4 WS 연결 실패

- producer 또는 consumer가 다른 streamId 사용 — 둘 다 `phenotyping/gimbal` 사용 확인
- relay plugin이 dev server에 mount 되어 있는지 — Vite startup log 확인

15.5 디텍션이 0건만 나옴

- 디버그 로그 활성 — `?debug=scenarios`
- 카메라가 식물을 보지 못하는 위치 — railX 값 확인
- HSV 파라미터가 너무 보수적 — `hsv_parameters.json` 의 sat threshold 낮춰서 시도

16. 코드 컨벤션 / 협업 흐름

16.1 코드 컨벤션

- TypeScript strict mode
- React functional components + hooks (class component 금지)
- Zustand store 사용 — direct `useState` 글로벌 사용 금지
- `console.*` 직접 호출 금지 — `createLogger` 사용 (`logger-enforcement.spec.ts` 로 검증)
- 좌표 변수는 좌표계 suffix 필수 (`worldX` , `plantLocalX` 등)

16.2 Pull Request 흐름

1. Feature branch에서 작업
2. 로컬에서 `npm run build` 통과
3. `npx playwright test tests/architecture/coordinate-contracts.spec.ts tests/architecture/logger-enforcement.spec.ts` 통과
4. PR 제출 + 리뷰

16.3 핵심 SSOT 문서

영역	SSOT
코딩 가이드	`CLAUDE.md`
좌표계	`docs/architecture/COORDINATE_SYSTEMS.md`
Mesh Anchor	`docs/architecture/MESH_ANCHORS.md`
토마토 데이터	`packages/tomato-engine/models/INDEX.jsonc`
로깅	`docs/architecture/LOGGING.md`
시험 정책	`tests/architecture/PROBES.md`